

Assignment 2: What Does an Operation Cost?

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 24, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus). Answer every question. Show all working for Numerical and Design questions. Every question uses the array convention from lecture — `A[0..n-1]`, 0-based indexing — and every complexity claim you make must name its case (best/worst/average) and its operation, exactly as required in class.

2.1 Conceptual

Q1. A bound must always name its case, and $O(n)$ is a class of functions, not an exact step count — two habits the lecture insisted on. For each statement below, say whether it is correctly stated (good) or incomplete/incorrect (bad), and justify in one sentence.

- (a) “Merge sort is $O(n \log n)$.”
- (b) “Linear search is $\Omega(1)$ in the best case.”
- (c) “Since $T(n) = O(n^2)$, the algorithm takes exactly n^2 steps on every input.”
- (d) “The nested-loop duplicate check from lecture is $\Theta(n^2)$ in every case.”

Q2. Binary search is only *correct* when the array is sorted — this is a precondition, not an option to be checked later. Consider the *unsorted* array `A = [30, 5, 47, 12, 68, 3, 90]` (so $n = 7$), and run `binary_search` (as given in lecture) for `key = 47`, which *is* present, at index 2.

- (a) Trace the algorithm pass by pass, in the same `lo/hi/mid/A[mid]` table format used in lecture, until it terminates.
- (b) State what the algorithm returns, and say whether this is the correct answer for `key = 47` on this array.
- (c) In two or three sentences, explain *why* this happens, in terms of what the sorted precondition is supposed to guarantee about discarding a half of the array at each step.

2.2 Numerical

Q3. The calculator’s symbol table (this week’s running example) is, for this question, the array `A = [42, 17, 90, 8, 55, 3]` (so $n = 6$). Counting the comparison `A[i] == key` as in lecture, run `linear_search` on this array for each of the three keys below.

- (a) `key = 42`. State the number of comparisons performed, and which case (best/worst/average) this run witnesses.
- (b) `key = 3`. State the number of comparisons performed, and which case this run witnesses.
- (c) `key = 99` (not present in the array). State the number of comparisons performed, and which case this run witnesses.
- (d) Using the average-case formula derived in lecture for a successful search with `key` equally likely at any position, compute the average-case comparison count for $n = 6$, and state in one sentence what this number represents.

Q4. Consider the sorted array `A = [2, 9, 14, 23, 31, 40, 48, 56, 63, 71]` (so $n = 10$), and run `binary_search` (as given in lecture) for `key = 40`.

- (a) Produce a pass-by-pass table (`pass`, `lo`, `hi`, `mid`, `A[mid]`, comparison result), exactly as in lecture, showing every step of `lo`, `hi`, and `mid` until the search terminates.
- (b) State the number of passes the search takes.
- (c) Compute $\lceil \log_2 10 \rceil$, and compare it to your answer in (b). In one or two sentences, explain the relationship between this number and the worst-case bound for binary search on an array of this size.

2.3 Design

Q5. Consider the following C function, which the lecture did *not* analyse:

```
int array_max(int A[], int n) {
    int max = A[0];
    for (int i = 1; i < n; i++)
        if (A[i] > max)
            max = A[i];
    return max;
}
```

- (a) State the basic operation, and derive the *exact* cost function $T(n)$ by counting how many times it executes. Your count should hold for *every* input array of size n , not just one example — explain why the comparison `A[i] > max` runs the same number of times regardless of what values are in `A`.
- (b) State the best case, worst case, and average case of `array_max`, each as a comparison count. In one or two sentences, explain why all three coincide here, in contrast to `linear_search`, where they do not.
- (c) Show that your $T(n)$ from part (a) is $\Theta(n)$: exhibit valid constants c_1 , c_2 , and n_0 for the lower and upper bounds respectively, and verify both inequalities, following the method used in lecture for $T(n) = 3n + 2$.
- (d) State the auxiliary space complexity $S(n)$ of `array_max`, and justify it by naming every local variable the function uses.

Q6. Consider the following C code, which counts something using a nested loop with *different* bounds from the duplicate-check loop analysed in lecture:

```
int count = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j <= i; j++)
        count++;
```

- (a) State the basic operation, and state how many times the inner loop executes for a fixed value of `i` (in terms of `i`).
- (b) Derive the exact closed-form cost function $T(n)$ by summing your answer to (a) over every value of `i` from 0 to $n - 1$, showing the index-substitution step, in the same style as the lecture's derivation of $\sum_{i=0}^{n-2} (n - i - 1)$.
- (c) State the asymptotic class of $T(n)$ (as $\Theta(\cdot)$ of some function of n), and justify it by identifying the dominant term of your closed form from (b).
- (d) State which case (best, worst, or average) this Θ bound applies to, and justify your answer by explaining whether `count`'s final value — and therefore the number of loop iterations — can ever differ between two runs with the same n .